

BUGLOG.md (schema below)

B## | symptom | root cause | fix | prevention rule

B08 | Pre-push repo audit found `packing.py` TRACKED by git, contradicting R024/EVIDENCE.md's own claim ("appeared untracked in the project root"; "`git log --all -- packing.py` shows zero commits ever touched it"; "never part of this repo"). | R024 was accurate when written. Some later commit's `git add -A/git add .` (not identified by message — the file's presence has no connection to any commit's stated purpose) swept the untracked third-party file into the index without anyone noticing, since its presence causes no test or checkpoint to fail. `git log --all -- oneline -- packing.py` now shows one commit that touched it. | `git rm --cached packing.py` (file kept on disk, unmodified) restores the untracked state R024 describes; added to `.gitignore` with a comment citing R024, so a future blanket `git add` cannot repeat this. | **A "never part of this repo" claim is a claim about the CURRENT git index, and drifts silently the moment anyone runs a blanket git add.** Any RESULTS/EVIDENCE row asserting absence (untracked, unused, never-touched) should be spot-checked with `git ls-files/git log --all` before a push or audit, not trusted from when it was written — this is the same lesson as the stale-count findings from the C4 audits, applied to git state instead of prose. B02 | `c1.md` says "after Checkpoint 5, submit the small IBM job within budget so it runs overnight" — cannot proceed. | No `qiskit_ibm_runtime` package installed in `.venv`, and no IBM Quantum API token/account configured anywhere on this machine (checked `env vars`, `~/.qiskit*`, common config dirs). | RESOLVED: user supplied an IBM Cloud API key at `apikey.json` (gitignored, never committed, never printed). Installed `qiskit-ibm-runtime==0.49.0`; connected with `channel="ibm_cloud"` (the legacy `ibm_quantum` channel is gone as of this IBM account/plan — "open"/free instance "Martin"); saved the account locally via `QiskitRuntimeService.save_account(...)` (persists to `~/.qiskit/`, outside the repo). Submitted a 2-circuit/4000-shot job (see RESULTS R008) via `SamplerV2` from inside the live, already-validated notebook kernel (reused `build_shadow_hadamard_circuit/HAM/sub_seed` unmodified — no reimplementation). | CONVENTIONS.md does not define a specific hardware-job shot budget (only the 256,000-shot simulator budget). Judgment call: `t=0.9` (matches Checkpoint 5), one representative basis `[X,Y,Z]`, `method="trotter"` `reps=1` (exact/UnitaryGate is simulator-only per CONVENTIONS §2), 2000 shots/quadrature — small enough to be a good-faith overnight test, not a statistically-complete result. Do not treat R008 as a reportable number until the job completes and is cross-checked against `exact_ref`; if it doesn't complete before the 13:00 Aug 13 go/no-go, CLAUDE.md's fallback applies: drop hardware, bonus 11 noise model becomes the fallback, never mention hardware again. B07 | Track B's statevector validation failed at `3.31e-01` (want `<1e-10`) on first run, despite the derivation being correct -- and `chi_AB(0)=1` came out right, which made it look like only the time-dependence was broken. | The REFERENCE was wrong, not the circuit. `Operator(qc)` on a circuit containing an un-synthesised `PauliEvolutionGate(H, t, synthesis=SuzukiTrotter(...))` does NOT return the Trotterised unitary -- Qiskit defers synthesis to transpile time, so `Operator()` evaluates the gate's exact mathematical definition and hands back `exp(-iHt)` itself. Confirmed numerically: my "Trotter `reps=1`" reference matched `exact_unitary` to **8.3e-16**, i.e. exactly zero Trotter error, which is impossible for `reps=1` (Checkpoint 3a measures 0.82 operator error there). So I was comparing the circuit's genuinely-Trotterised `W` against a reference that had silently become the exact `U`. | Replaced the reference with `trotter_unitary()`, which extracts `W` from the `|1><1|` block of the very gate `build_controlled_evolution` produces -- the reference and the circuit now cannot diverge by construction. Identity then confirmed at **8.99e-15**. | This is the same "Qiskit defers synthesis" hazard the notebook already warns about for `.control()` ordering (§3.1), reappearing in a different guise: it also defers for `Operator()`. Rule: to obtain a *synthesised* unitary, transpile first, or extract it from an object that has already been transpiled -- never assume `Operator()` on a high-level gate reflects the synthesis you requested. Sanity-check that a "approximate" reference actually carries nonzero approximation error before trusting it. B06 | Track E's purity-based consistency witness reported a -2.2 sigma gap ON THE IDEAL SIMULATOR, where the identity $\text{Tr}[(\rho^{\wedge}(I))^{\wedge}2] = (1+|\chi|^{\wedge}2)/2$ holds EXACTLY and the gap must therefore be zero. Worse, the real-hardware gap was -2.2 sigma too -- numerically indistinguishable, which would have made the whole witness look like it had no discriminating power. | Wrong uncertainty estimate, not a wrong point estimate. Purity is a U-statistic over PAIRS of shadow snapshots, and

I used `std(pair_values)/sqrt(n_pairs)`. That is invalid: pairs are not independent -- with N shots and 400k sampled pairs each shot appears in ~11 pairs, so the effective sample size is set by N (the shots), not by how many pairs I chose to draw. The sem was therefore far too small, inflating a statistically-null gap into an apparent 2.2 sigma effect. The point estimates were fine throughout. | Replaced with a 20-block jackknife: shuffle the shots (ESSENTIAL -- records arrive grouped by basis, so contiguous blocks would each contain a single basis and the estimator would be meaningless), split into disjoint blocks, estimate purity independently within each, and take the spread ACROSS blocks. Ideal-sim gap went -2.2 sigma -> **-0.1 sigma** (correct null); the noisy-sim control leg then correctly fires at -7.6 sigma (1x noise) and -17.2 sigma (5x), confirming real discriminating power. | **Always include a control leg where the answer is known to be null.** This bug was invisible from the hardware number alone -- it was only detectable because the ideal-simulator case, where the identity holds exactly, was run alongside and disagreed with theory. A witness validated only on data whose true answer is unknown cannot be trusted. Also: any pair/U-statistic estimator needs its error bar computed over the underlying independent units, never over the derived pairs. B05 | While diagnosing a user-reported "unrecognized arguments: --initial-layout" error, an accidental real hardware submission happened. | `hardware_run.py` has no dry-run mode. I ran `--submit --shots 1` intending only to test that `argparse` accepts `--initial-layout` (which it did -- the flag was never actually broken, see resolution). There is no `--dry-run/--no-submit` guard, so testing the CLI end-to-end IS submitting. | Not a code bug to fix in the sense of broken behaviour -- `--initial-layout` works exactly as documented (verified: pinned `initial_layout`: [125, 126, 127, 137] printed correctly). The real issue is diagnostic: the user's reported error could not be reproduced against the committed script, git-log-confirmed at commit 011f454, with `--help` correctly listing the flag. Likely cause: a stale copy, a different working directory, or `--initial_layout` (underscore) typed instead of `--initial-layout` (hyphen) -- `argparse` does not alias the two. Logging job `d9umah0u5hac73ahdof0` (2 shots total, negligible) here rather than omitting it. | Add a `--dry-run` flag to `hardware_run.py` that builds and transpiles circuits, prints the same diagnostics, and exits before `SamplerV2(...).run(...)` -- so a CLI test can never accidentally submit again. B04 | R008 reported `"_rho^(I) = +1.5945 vs exact +2.2675, 16.7 sigma"` from the hardware job and blamed hardware noise. The number was measuring a different observable entirely. | MY ANALYSIS ERROR, not a hardware or notebook defect. The hardware job used ONE fixed shadow basis [X,Y,Z] (chosen to keep the job small). `pauli_snapshot_values` implements $\hat{P} = 3^w * \prod(s_j) * 1[b_j == P_j]$, which is unbiased ONLY when the bases are drawn i.i.d. uniform per shot -- that is what supplies the $1/3$ matching probability the 3^w factor compensates for. With a fixed basis the indicator is deterministic: for $Q = Z0+Z1+Z2$ under basis [X,Y,Z], the Z0 and Z1 terms can never match and contribute structurally zero, so the estimator returns 3 *rather than* 1. I applied a random-basis estimator to fixed-basis data and read the discrepancy as decoherence. | Withdrawn the 'line from R008 and annotated the correction inline. Verified the diagnosis on the NOISELESS simulator (`scratchpad/check_single_basis_bias.py`): random bases give `+2.2729+-0.0132` (0.4 sigma from exact 1, correct); fixed basis [X,Y,Z] gives `+2.6970+-0.0066`, matching 3 -- exact = `+2.6964` to 4 decimal places and sitting 65.4 sigma from 1 with zero noise present. The chi figures in R008 are unaffected -- they read the ancilla only and are basis-independent -- so the headline hardware finding (survival 0.179, 35.2 sigma) stands. | **Any fixed-basis job can report chi and nothing else.** A shadow-derived SYSTEM observable requires the random-basis ensemble (≤ 27 circuits per setting), which no hardware job here has run. This applies to the three pending R018 jobs as well: when they return, report chi only. Prevention: `hardware_run.py` must refuse to print shadow system observables unless run in a random-basis mode, and its `--fetch` output must state the basis it assumed. B03 | Overnight hardware job (R008) came back DONE but wildly wrong: `chi(0.9) 35.2 sigma` off on the real quadrature, `'_rho^(I) 16.7 sigma` off. | Not an estimator bug (verified: same `parse_memory/estimate_hadamard_signal/estimate_system_observable` code that passed Checkpoints 4-6 on the simulator; basis, quadrature order and circuit indices double-checked against the submission script). Most likely root cause: circuit depth. Trotter `reps=1` at `n=3` transpiles to depth 1729 / 435 CZ gates on `ibm_marrakesh`'s native gate set -- CONVENTIONS §2's own callout ("the exact 4-qubit UnitaryGate synthesises into FEWER CX gates than the Trotter circuit needs" at `n=3`) already flagged this circuit-size inversion; at typical superconducting 2q fidelities, ~435 sequential CZ gates plausibly decoheres the state most of the way to maximally mixed before the final measurement, which is consistent with the near-collapsed chi value observed. | RESOLVED (characterized, per user "go" decision at the 13:00 go/no-go): ran the given-complete Challenge 11 noise model (unmodified) at its default parameters (`p1=3e-4, p2=6e-3, p_ro=1.2e-2`) on the SAME `method="trotter" reps=1` circuit

family and measured its simulated damping ratio -- 0.342 (R014). The REAL hardware damping ratio is 0.179 (R008/R014), roughly 2x worse than this generic toy model predicts. Conclusion recorded as R014: real ibm_marrakesh noise at this depth exceeds a reasonable generic depolarizing+readout baseline -- an honest, quantified robustness finding, not an unexplained bad number. Left as-is; no further mitigation attempted this session (would need real per-gate calibration data from the backend, out of scope for tonight). B01 | `pip install -r requirements.txt` fails: `numpy==2.5.2` and `scipy==1.18.0` require Python `>=3.12`, which is not installed on this machine (available: system 3.10, anaconda 3.11). | Environment pin mismatch, not a code/convention bug — this machine never had Python 3.12. | Created `.venv` with anaconda's Python 3.11.5; installed `qiskit==2.5.1` and `qiskit-aer==0.17.2` exactly as pinned (both available for 3.11), but relaxed `numpy` to 2.4.6 (`<` pinned 2.5.2) and `scipy` to 1.17.1 (`<` pinned 1.18.0) — nearest available versions compatible with qiskit 2.5.1 on Python 3.11. `matplotlib==3.11.1` and `pylatexenc==2.11` installed exactly as pinned. | Not a CONVENTIONS.md law violation (§6 pins are tooling, not science) but flagged per the zero-improvisation rule since it deviates from the frozen environment doc. If Python 3.12 becomes available, recreate `.venv` with the exact pins and re-run Gate 0 + all checkpoints to confirm no numerical drift from the `numpy/scipy` delta (none expected — both are patch/minor bumps within the same major series)."""